

Q-Sec: Simulador Interactivo del Protocolo de Distribución Cuántica de Claves BB84

Abstract

El presente trabajo describe el diseño e implementación de Q-Sec, una aplicación web educativa que simula el protocolo de Distribución Cuántica de Claves (QKD) BB84. Motivado por la amenaza que representa la computación cuántica para los esquemas criptográficos clásicos, el trabajo busca ofrecer una herramienta accesible que permita comprender, de forma interactiva, cómo la mecánica cuántica garantiza la seguridad en el intercambio de claves y la detección de espionaje. El sistema se desarrolló en Python bajo una arquitectura en tres capas (presentación, negocio y datos), utilizando Flask para la interfaz web, la librería Qiskit de IBM para simular la preparación, transmisión y medición de qubits, y SQLAlchemy sobre SQLite para la persistencia de usuarios y resultados. La capa de negocio implementa la regla central del protocolo: la detección de un espía mediante el cálculo de la tasa de error cuántico (QBER) sobre una muestra de la clave filtrada, validada mediante pruebas unitarias con Pytest. Como resultado se obtuvo una aplicación funcional que permite configurar y ejecutar simulaciones completas del protocolo, visualizar sus resultados paso a paso y consultar un historial de sesiones, ofreciendo una alternativa didáctica frente a las herramientas existentes.

Palabras Clave: criptografía cuántica; distribución cuántica de claves; protocolo BB84; Qiskit; simulación educativa; ciberseguridad.

1. Introducción

En un contexto donde la seguridad de las comunicaciones digitales es cada vez más crítica, los esquemas criptográficos clásicos —como RSA— enfrentan una amenaza concreta ante el avance de la computación cuántica. Algoritmos como el de Shor permitirían, en un escenario con hardware cuántico suficientemente potente, factorizar números primos grandes en tiempo polinomial, comprometiendo buena parte de la infraestructura criptográfica actual. Frente a este escenario, la criptografía cuántica propone soluciones cuya seguridad no depende de la dificultad computacional de

un problema matemático, sino de las leyes de la física.

El protocolo BB84, propuesto por Bennett y Brassard en 1984 [1], es el ejemplo pionero de Distribución Cuántica de Claves (QKD). Permite a dos partes —tradicionalmente llamadas Alice y Bob— establecer una clave secreta compartida, con la particularidad de poder detectar la presencia de un espía (Eve) que intente interceptar la comunicación, gracias al principio de incertidumbre de Heisenberg y al teorema de no clonación cuántica.

A pesar de la solidez teórica del protocolo, su comprensión suele resultar poco intuitiva para quienes se acercan por primera vez a la computación cuántica, dado que requiere manejar conceptos como superposición, bases de medición y colapso del estado cuántico. Existen herramientas didácticas que buscan reducir esta barrera de entrada, aunque, como se discute en la Sección 3, la mayoría se limita a visualizaciones esquemáticas o a scripts de ejecución local, sin persistencia de resultados ni una interfaz orientada al usuario final.

Este trabajo presenta Q-Sec, un sistema web que simula el protocolo BB84 de punta a punta, permitiendo a los usuarios ejecutar simulaciones configurables, visualizar los resultados paso a paso y almacenar un historial de sus sesiones. El objetivo es ofrecer una herramienta que combine rigor técnico —mediante el uso de un simulador de circuitos cuánticos real— con una experiencia de uso simple, pensada para fines educativos. El resto del trabajo se organiza de la siguiente manera: la Sección 2 describe el diseño e implementación del sistema; la Sección 3 discute trabajos relacionados; y la Sección 4 presenta las conclusiones y las líneas de trabajo futuro.

2. Diseño e Implementación del Sistema

2.1. Arquitectura

Q-Sec fue diseñado bajo una arquitectura en tres capas, buscando una separación clara de responsabilidades que facilite el mantenimiento y la extensión del sistema (Figura 1). La capa de presentación se implementó con Flask, y expone las rutas HTTP de la aplicación (registro, autenticación, configuración y ejecución de simulaciones, visualización de resultados e historial), utilizando plantillas Jinja2 y formularios validados con WTForms. La

capa de negocio concentra la lógica central del sistema: la implementación del protocolo BB84, el controlador de autenticación y el controlador de simulaciones, donde se validan las reglas de negocio. La capa de datos gestiona la persistencia mediante el ORM SQLAlchemy sobre una base de datos SQLite, a través de dos modelos principales —Usuario y Sesión de Intercambio (Figura 2)— y sus respectivos repositorios, que encapsulan el acceso a los datos. Esta separación permitió, por ejemplo, validar de forma aislada la lógica del protocolo BB84 sin depender del framework web.

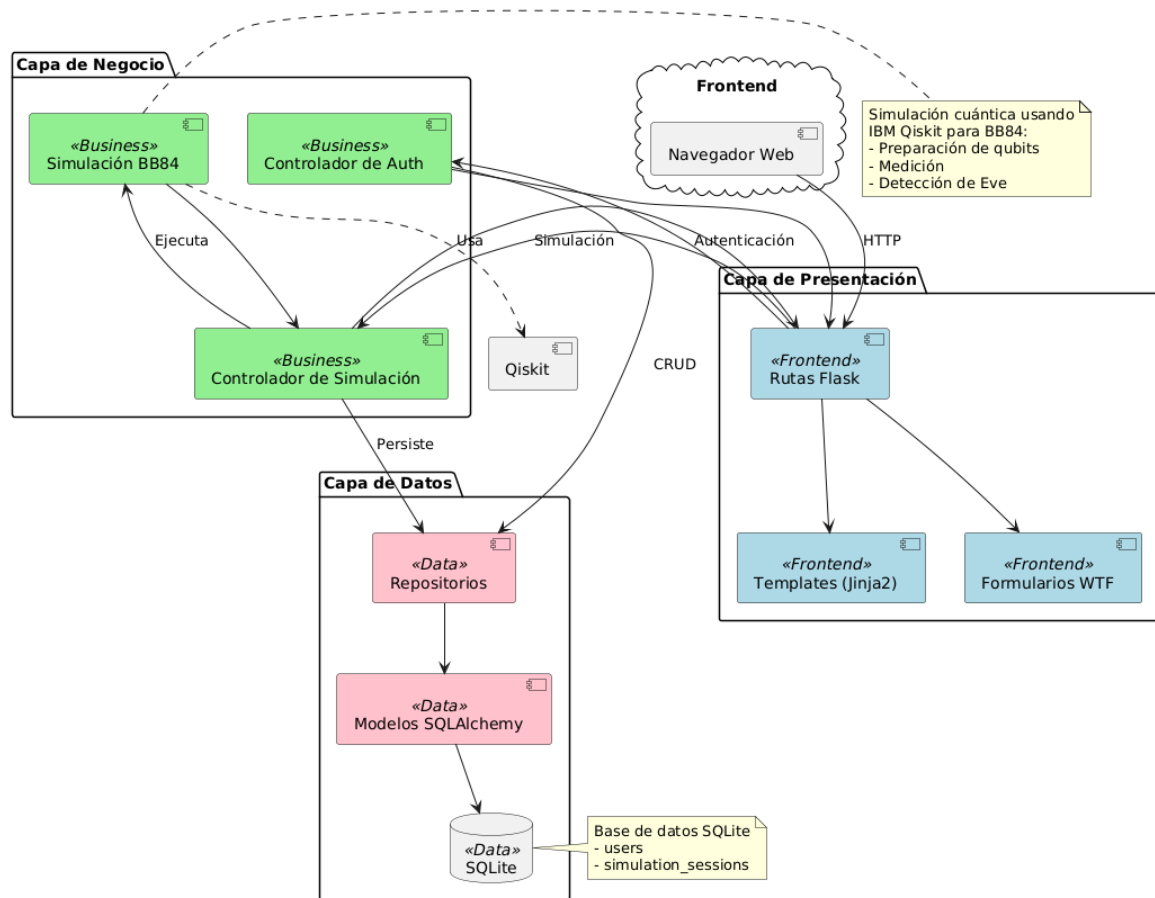


Figura 1. Arquitectura en tres capas de Q-Sec.

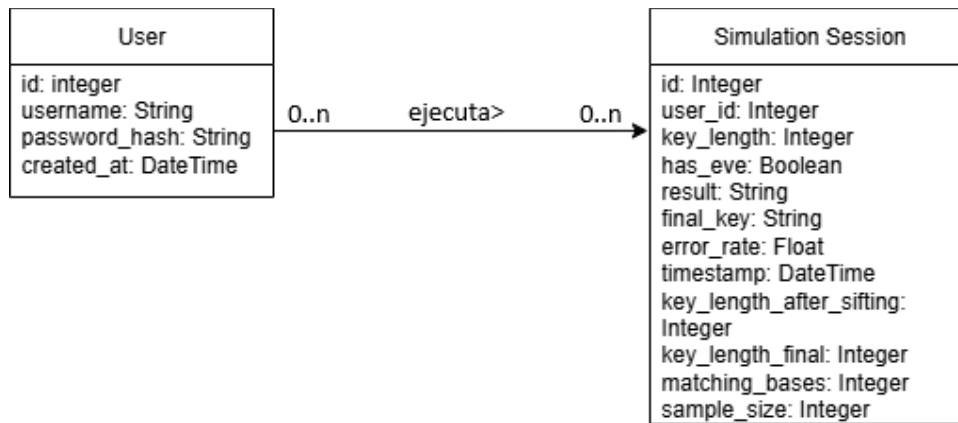


Figura 2. Modelo de dominio de Q-Sec.

2.2. Simulación del protocolo BB84

La implementación del protocolo utiliza Qiskit [2] y su backend de simulación qasm_simulator (Qiskit Aer) para representar cada qubit intercambiado entre Alice y Bob mediante un circuito cuántico de un único qubit. El proceso sigue los pasos estándar del protocolo:

- 1) Alice genera una secuencia de n bits aleatorios y una secuencia de n bases aleatorias (rectilínea + o diagonal ×).
- 2) Cada bit se codifica en un circuito cuántico: se aplica una puerta X si el bit es 1, y una puerta Hadamard si la base elegida es diagonal.
- 3) (Opcional) Si la simulación incluye un espía, Eve intercepta cada qubit, lo mide con una base propia elegida al azar y prepara un nuevo qubit con el resultado obtenido, reenviándolo a Bob; esta operación introduce errores adicionales cuando la base de Eve no coincide con la de Alice.
- 4) Bob elige sus propias bases aleatorias, independientes de las de Alice, y mide cada qubit recibido.
- 5) Alice y Bob comparan públicamente —y solo— las bases utilizadas (nunca los bits), descartando las posiciones donde no coincidieron, para obtener la clave tamizada.
- 6) Sobre una muestra de la clave tamizada (25%, con un máximo de 20 bits) se calcula la tasa de error cuántico (QBER), comparando los bits de Alice y Bob en esas posiciones.
- 7) Si la QBER es menor a un umbral de seguridad (11%), la clave se considera segura y se descartan los bits usados en la verificación para conformar la clave final; en caso

contrario, se asume la presencia de un espía y la sesión se marca como comprometida.

Este umbral no es arbitrario: en un ataque de interceptación-reenvío como el implementado, la intervención de Eve introduce estadísticamente una tasa de error cercana al 25% en la clave tamizada, muy por encima del ruido típico de un canal sin intervención (habitualmente inferior al 5%). El valor de 11% utilizado en Q-Sec corresponde a la cota de seguridad incondicional demostrada por Shor y Preskill [6] para el protocolo BB84, por debajo de la cual aún es posible garantizar la extracción de una clave segura mediante técnicas de reconciliación de la información y amplificación de privacidad, que exceden el alcance de este trabajo y se retoman en la Sección 4.

La Tabla 1 resume los requerimientos funcionales que guiaron la implementación.

Req.	Descripción
RF01	Registro y autenticación de usuarios.
RF02	Inicio de una nueva sesión de intercambio de clave.
RF03	Configuración de longitud de clave y presencia de espía.
RF04	Ejecución completa del protocolo BB84 (preparación, transmisión, medición, comparación de bases y verificación de error).
RF05	Persistencia del resultado de cada sesión en base de datos.
RF06	Consulta de historial de sesiones previas.

Tabla 1. Requerimientos funcionales principales.

2.3. Persistencia y experiencia de usuario

Cada sesión de intercambio ejecutada por un usuario autenticado se persiste en la base de datos junto con su configuración (longitud

de clave solicitada, presencia o no de Eve), sus resultados intermedios (bits coincidentes tras la comparación de bases, tamaño de la muestra utilizada para el cálculo de QBER) y su resultado final (clave segura o comprometida, tasa de error, clave resultante cuando corresponde). Esto permite al usuario consultar un historial completo de sus simulaciones y comparar resultados entre distintas configuraciones. La autenticación utiliza Flask-Login junto con hashing de contraseñas mediante Werkzeug, evitando el almacenamiento de contraseñas en texto plano; la gestión de sesión se realiza mediante cookies firmadas en lugar de variables de estado local, lo que permite que la aplicación funcione de forma independiente en distintas pestañas o navegadores del mismo equipo.

2.4. Validación

La corrección de la implementación se validó mediante un conjunto de pruebas automatizadas con Pytest, organizadas en cuatro módulos: pruebas del protocolo BB84 (verificando que la longitud de la clave tamizada es consistente con las bases coincidentes y que la presencia de Eve incrementa significativamente la tasa de error detectada), pruebas de los modelos de datos, pruebas de integración entre capas y pruebas de las rutas de la aplicación web. Esta suite permite ejecutar la simulación repetidas veces bajo distintas configuraciones y confirmar estadísticamente que la tasa de detección de espionaje se comporta según lo esperado.

3. Trabajos Relacionados

La simulación educativa de protocolos de distribución cuántica de claves no es un problema nuevo, y existen antecedentes tanto en el ámbito académico como en el desarrollo de herramientas de código abierto.

Kohnle y Rizzoli [3] desarrollaron, en el marco del proyecto QuVis (Quantum Mechanics Visualisation Project), un conjunto de cuatro simulaciones interactivas para distintos protocolos de QKD, incluido

BB84, orientadas específicamente a la enseñanza universitaria y evaluadas empíricamente con estudiantes durante tres años. A diferencia de Q-Sec, estas simulaciones priorizan la visualización conceptual del protocolo mediante fotones polarizados o partículas de espín $1/2$, sin apoyarse en un framework de computación cuántica real ni ofrecer persistencia de resultados o gestión de usuarios.

Por otro lado, distintos trabajos abordaron la implementación de BB84 directamente sobre Qiskit. Saeed et al. [4] presentaron una implementación del protocolo mediante circuitos cuánticos ejecutados tanto en simuladores locales como en hardware cuántico real de IBM, enfocada en validar la viabilidad de una propuesta de circuito para BB84 más que en construir una herramienta interactiva de uso final. De forma similar, Adu-Kyere et al. [5] modelaron y simularon el protocolo en Python, evaluando su comportamiento bajo distintos escenarios de ruido y ataque, también en forma de script de análisis y no como aplicación web.

Existen además numerosas implementaciones de código abierto que replican el protocolo BB84 sobre Qiskit con distintos niveles de complejidad, generalmente en forma de notebooks o scripts de línea de comandos pensados para uso individual, sin persistencia de resultados ni gestión de usuarios.

Q-Sec se diferencia de estos antecedentes en que combina una simulación cuántica real — mediante Qiskit, y no una mera visualización esquemática— con una arquitectura de aplicación web completa, que incluye autenticación de usuarios, persistencia de resultados y un historial consultable de simulaciones. Esto la posiciona como una herramienta más cercana a un producto de software educativo que a un script de demostración, aunque a costa de no implementar aún las etapas de reconciliación de la información y amplificación de privacidad que sí contemplan otros trabajos orientados al análisis de seguridad del protocolo.

4. Conclusión y Trabajos Futuros

Este trabajo presentó Q-Sec, una aplicación web que simula el protocolo BB84 de Distribución Cuántica de Claves, integrando una simulación cuántica realizada con Qiskit dentro de una arquitectura de software en tres capas, con persistencia de resultados y una interfaz orientada a usuarios sin conocimientos previos de computación cuántica. La validación mediante pruebas automatizadas mostró que la implementación reproduce correctamente el comportamiento esperado del protocolo, tanto en ausencia como en presencia de un espía.

El trabajo presenta, no obstante, limitaciones que delimitan su alcance. En primer lugar, la simulación se ejecuta sobre un simulador de circuitos cuánticos (Qiskit Aer) y no sobre hardware cuántico real, por lo que no captura efectos de ruido o decoherencia propios de una implementación física, tal como sí lo abordan trabajos como el de Adu-Kyere et al. [5]. En segundo lugar, el sistema no implementa las etapas posteriores a la detección de espionaje que forman parte de un esquema de QKD completo, como la reconciliación de la información o la amplificación de privacidad; actualmente, ante una tasa de error aceptable, la clave final se construye descartando simplemente los bits usados en la muestra de verificación. Finalmente, el modelo de espionaje implementado se limita a un ataque de interceptación-reenvío básico, sin contemplar estrategias más sofisticadas.

Como trabajo futuro se plantea: (i) incorporar un modelo de ruido de canal configurable, para distinguir mejor los efectos del ruido físico de los de un ataque real; (ii) implementar las etapas de reconciliación de la información y amplificación de privacidad, completando el esquema de QKD; (iii) permitir la ejecución de la simulación sobre hardware cuántico real a través de IBM Quantum, comparando los resultados obtenidos frente al simulador; y (iv) incorporar visualizaciones interactivas paso a paso del estado de cada qubit, en línea

con el enfoque didáctico de herramientas como QuVis [3], para reforzar la comprensión conceptual del protocolo más allá de sus resultados numéricos.

Referencias

- [1] C. H. Bennett y G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," en Proc. IEEE Int. Conf. on Computers, Systems and Signal Processing, Bangalore, India, 1984, pp. 175-179.
- [2] Qiskit contributors, "Qiskit: An Open-source Framework for Quantum Computing," 2024. DOI: 10.5281/zenodo.2573505.
- [3] A. Kohnle y A. Rizzoli, "Interactive simulations for quantum key distribution," European Journal of Physics, vol. 38, no. 3, art. 035403, 2017.
- [4] M. H. Saeed, H. Sattar, M. H. Durad y Z. Haider, "Implementation of QKD BB84 Protocol in Qiskit," en 2022 19th International Bhurban Conference on Applied Sciences and Technology (IBCAST), Islamabad, Pakistan, 2022, pp. 689-695.
- [5] A. Adu-Kyere, E. Nigussie y J. Isoaho, "Quantum Key Distribution: Modeling and Simulation through BB84 Protocol Using Python3," Sensors, vol. 22, no. 16, art. 6284, 2022.
- [6] P. W. Shor y J. Preskill, "Simple proof of security of the BB84 quantum key distribution protocol," Physical Review Letters, vol. 85, no. 2, pp. 441-444, 2000.